SLAYER SECURITY

SMART CONTRACT

SECURITY REVIEW

PREPARED FOR

AMMO MARKETS

PREPARED BY

SLAYER SECURITY

MAY, 2026

1. About Slayer Security

Slayer Security is a specialist smart contract security practice providing smart contract audits, DeFi security reviews, and rapid pre-launch assessments. The team approaches every codebase as an adversary would, hunting for exploitable bugs rather than checking boxes. Slayer Security combines manual review with AI-assisted analysis, transparent reporting, and actionable findings intended to help teams ship securely. Check our work at https://slayer-security.xyz/porfolio.

2. Disclaimer

This review is a point-in-time assessment of the in-scope smart contracts and should not be interpreted as a guarantee that the system is free of vulnerabilities. The assessment was performed under practical limits, including the agreed scope, available review time, and the state of the code at the time of analysis. While the review aims to identify meaningful security issues, some risks may remain undiscovered or may arise from future changes, integrations, or deployment conditions. Continued testing, monitoring, and follow-up audits and bug bounties are recommended as part of an ongoing security process.

3. About Ammo Markets

Ammo Markets runs on Avalanche and wraps real-world ammunition into ERC20s, one token per caliber and load type. Each token represents one physical round, backed 1:1 by inventory. The on-chain protocol is the buy/sell and accounting layer,the physical ammunition itself sits with the custodian.

To mint, a user locks USDC, pays a small mint fee, and a keeper finalizes the order at the oracle price. Holders can later redeem tokens for physical ammunition or cash out to USDC using exit liqudity feature. Redemptions carry a user-selected deadline, and unfulfilled orders can be cancelled by the user as a self-rescue path.

Spot prices come from an on-chain oracle that keepers maintain on a regular cadence.

Trades on listed DEX pools pay a transfer tax that the protocol swaps to AVAX for the treasury. Liquidity providers can stake LP in a farm where emissions split evenly across active pools and taper off over time, under hard caps on total minting.

4. Scope

This engagement was conducted in two phases:

1. Phase - 1 — Initial review
2. Phase - 2 — Fixes verification and review of updated contracts

Phase - 1 (Initial review)

Review commit: 977927ce743abbde7a013e4a3a2efb6f57bdc5a3

In-scope contracts

- `src/AmmoManager.sol`
- `src/AmmoFactory.sol`
- `src/CaliberMarket.sol`
- `src/AmmoToken.sol`
- `src/ExitLiquidityPool.sol`
- `src/PriceOracle.sol`
- `src/AmmoPriceFunctions.sol`
- `src/ProtocolToken.sol`
- `src/ProtocolEmissionController.sol`
- `src/AmmoMarketLPFarm.sol`
- `src/AmmoLiquidityManager.sol`

Phase - 2 (Fixes verification and updated contracts)

Phase - 2 re-reviewed the codebase after fixes were delivered for Phase - 1 findings and covered additional protocol updates made during the engagement.

Review commits: df4e5c22ac60b53a20512f201faf073bb58409fd and ac2302d7d1ff73f2eb949ca38d86798a25484b04

In-scope contracts

- `src/AmmoFactory.sol`
- `src/AmmoLiquidityManager.sol`
- `src/AmmoManager.sol`
- `src/CaliberToken.sol`
- `src/CaliberMarket.sol`

- `src/PriceOracle.sol`
- `src/external` (includes fork of Pharoah contracts + `GvToken`)

Token naming

Between the two phases, the protocol renamed its per-market ERC20 from `AmmoToken` to `CaliberToken`. Each `CaliberMarket` deploys its own `CaliberToken.sol`. In this report, Phase - 1 references to `AmmoToken` / `AmmoToken.sol` describe the same asset that Phase - 2 calls `CaliberToken`.

In some place in phase-1 report we have intentionally changed naming AMMO to CALIBER for ease of understanding.

5. Security Model and Trust Assumptions

Trusted Roles

The `owner`, `guardian`, and `keeper` roles are assumed to be fully trusted.

These roles are expected to act honestly and in the best interest of the protocol. Malicious behavior, key compromise, or incorrect privileged actions by these roles are outside the scope of this report unless explicitly stated otherwise.

Non-Cancellable Mint Requests With Zero Deadline

When a user calls `startMint` with `deadline = 0`, the mint request does not expire and cannot be cancelled by the user.

In this case, the request depends on the keeper or protocol operation flow to either finalize the mint or cancel it on the user's behalf. This is treated as an expected design choice.

Token Denylist

CALIBER tokens include a denylist feature controlled through `AmmoManager`.

Denied addresses cannot send or receive CALIBER tokens. This is treated as an intentional token design choice and depends on trusted owner or guardian operation. Misuse of the denylist, incorrect denylist configuration, or compromise of the privileged roles that control it are considered part of the trusted-role assumption.

Equal Per-Pool LP Farm Emissions

`AmmoMarketLPFarm` splits emissions equally across every active pool with nonzero stake, without weighting by staked USD value, DEX liquidity depth, or trading

demand. A pool with less total stake can therefore offer higher reward APR per dollar deposited, which may pull LP capital toward thinner pools and away from markets that need greater depth.

This is treated as an intended emission model. Operators are expected to manage which pools are active and monitor how liquidity is distributed across markets.

Dust Stakes in Rewardable Pools

Any active pool with `totalStaked > 0` is counted as a full rewardable pool in the equal per-pool split, regardless of how small the stake is. Very small stakes can increase `rewardablePoolCount`, dilute emissions for pools with meaningful TVL, and let the dust staker harvest nearly the entire per-pool share from otherwise empty pools.

This is treated as an operational design choice. Pool activation, minimum liquidity expectations, and ongoing farm configuration are expected to be managed through trusted governance or operational process.

6. Findings Summary

Phase - 1

ID	Severity	Title	Component
H-01	High	Expired Mint Orders Can Be Used to Brick Daily Minting	`CaliberMarket.sol`
M-01	Medium	Pool-Based CALIBER Taxes Break Standard Pharaoh Router Integrations	`AmmoToken.sol`
M-02	Medium	Out-of-Order Fulfillments Can Overwrite Fresh Oracle Prices	`AmmoPriceFunctions.sol`
M-03	Medium	Locked Request Pricing Can Be Abused Around Keeper Price Updates	`CaliberMarket.sol`, `PriceOracle.sol`
M-04	Medium	Permissionless Tax Sales Can Be Forced Through a Short TWAP Window	`AmmoToken.sol`
M-05	Medium	First Pool Updated Near Farm Cap Can Starve Other Rewardable Pools	`AmmoMarketLPFarm.sol`
L-01	Low	Refunded Mint Dust Is Still Counted Toward Protocol Token Release	`CaliberMarket.sol`, `ProtocolEmissionController.sol`
L-02	Low	Expired-User Authorization Uses Misleading Revert Errors	`CaliberMarket.sol`
L-03	Low	Emergency Withdraw Consumes Farm Cap Without Minting Rewards	`AmmoMarketLPFarm.sol`
L-04	Low	Dust Exits Can Create Unfinalizable Orders	`CaliberMarket.sol`, `ExitLiquidityPool.sol`

Phase - 2

ID	Severity	Title	Component
M-01	Medium	Non-Native Pair Liquidity Removal Can Bypass Effective Slippage Checks	`CaliberToken.sol`, router
L-01	Low	Denied Spender Can Move Tokens via Existing Allowances	`CaliberToken.sol`

7. Findings (Phase - 1)

[H-01] Expired Mint Orders Can Be Used to Brick Daily Minting

Summary

A user can fill the market's daily mint cap and cancel the order immediately to recover their USDC. Capacity consumed in `startMint` is not released on `cancelMint`, so other users cannot mint for the rest of the day. The attacker needs

no lasting capital cost.

Vulnerability Detail

`startMint()` accepts any `deadline`, including one that is already expired. There is no check that `deadline > block.timestamp`. The function consumes daily capacity and escrows USDC before the order can be finalized:

```
function startMint(uint256 usdcAmount, uint64 deadline) external returns (uint256 orderId
    ) {
    // ...
    _consumeDailyMintCapacity(usdcAmount);
    _safeTransferFrom(usdc, msg.sender, address(this), usdcAmount);

    orderId = nextMintOrderId++;
    mintOrders[orderId] = MintOrder({
        user: msg.sender,
        usdcAmount: usdcAmount,
        // ...
        deadline: deadline,
        status: OrderStatus.Requested
    });
}
```

`_consumeDailyMintCapacity` permanently increases `dailyMintUsedUsdc` for the current day:

```
function _consumeDailyMintCapacity(uint256 usdcAmount) internal {
    uint256 cap = manager.marketDailyMintCapUsdc(address(this));
    // ...
    uint256 newUsed = used + usdcAmount;
    if (newUsed > cap) revert DailyMintCapExceeded();

    dailyMintDay = day;
    dailyMintUsedUsdc = newUsed;
}
```

`cancelMint()` refunds the user in full but never releases the consumed capacity:

```
function cancelMint(uint256 orderId, uint8 reasonCode) external nonReentrant {
    MintOrder storage order = mintOrders[orderId];
    if (order.status != OrderStatus.Requested) revert InvalidStatus();
    _requireKeeperOrExpiredUser(order.user, order.deadline);

    order.status = OrderStatus.Canceled;
    _safeTransfer(usdc, order.user, order.usdcAmount);
}
```

A non-keeper can only cancel after the deadline has passed:

```
function _requireKeeperOrExpiredUser(address user, uint64 deadline) internal view {
    if (!manager.isKeeper(msg.sender)) {
        if (msg.sender != user) revert NotKeeper();
        if (deadline == 0) revert DeadlineNotSet();
        if (block.timestamp <= deadline) revert DeadlineExpired();
    }
}
```

An attacker can therefore call `startMint(cap, uint64(block.timestamp - 1))` and immediately call cancelMint(orderId, 0) in the same block (or atomically from a helper contract and using a flash loan), exhausting the daily cap while recovering all USDC.

Impact

- Minting on the affected caliber market is blocked until the next UTC day, where the malicious user can attack again, blocking for the next day as well. This can be continued as long as the attacker wishes
- Repeatable daily and across markets with separate caps.

Recommendation

- Reject mint orders with `deadline <= block.timestamp` in `startMint()`.
- On `cancelMint`, decrement `dailyMintUsedUsdc` when the order was created on the same day and never finalized (`dailyMintDay == order.createdAt / 1 days`).

Ammo: Fixed: commit ac2302d7d1ff73f2eb949ca38d86798a25484b04. A minimum deadline of 24 hours is enforced on `startMint`, and `cancelMint` now decreases the daily limit if the order is cancelled on the same day.

Slayer Security: Verified.

[M-01] Pool-Based CALIBER Taxes Break Standard Pharaoh Router Integrations

Summary

CALIBER charges transfer tax only when a transfer touches a configured taxed pool. Standard Pharaoh/Solidly router and UI flows quote swaps and liquidity additions from reserves and pair fees only, so they do not account for CALIBER's extra pool-based transfer tax. Normal CALIBER swap routes can therefore revert, under-deliver relative to the user's effective minimum output, or mint less LP than expected.

Vulnerability Detail

`AmmoToken` determines tax from the transfer endpoints. A buy is taxed when `from` is a configured pool, and a sell is taxed when `to` is a configured pool:

```
(uint256 buyBps, ) = manager.tokenPoolTax(address(this), from);
if (buyBps > 0) return (amount * buyBps) / _BPS_DIVISOR;

(, uint256 sellBps) = manager.tokenPoolTax(address(this), to);
if (sellBps > 0) return (amount * sellBps) / _BPS_DIVISOR;
```

Standard exact-input router swaps quote output from the full `amountIn`:

```
amounts = getAmountsOut(amountIn, routes);
amounts[i + 1] = IPair(pair).getAmountOut(amounts[i], routes[i].from);
```

The router then transfers the quoted input into the pair and executes the swap:

```
_safeTransferFrom(
    routes[0].from,
    msg.sender,
    pairFor(routes[0].from, routes[0].to, routes[0].stable),
    amounts[0]
);
_swap(amounts, routes, address(this));
```

For an CALIBER sell, the transfer is `user -> CALIBER/WAVAX pair`. Since `to` is the taxed pool, the pair receives only the net amount after sell tax, while the router still asks the pair to output the amount quoted for the gross input:

```
User input:   1,000 CALIBER
Sell tax:     3%
Pair input:   970 CALIBER
Router quote: output for 1,000 CALIBER
Result:       swap can revert because output is too high for actual input
```

For an CALIBER buy, the output transfer is `CALIBER/WAVAX pair -> user`. Since `from` is the taxed pool, the user receives the net amount after buy tax. The normal router path can satisfy `amountOutMin` against the pre-tax quoted amount while the user receives less than their intended minimum:

```
Router quote:  1,000 CALIBER
amountOutMin:    995 CALIBER
Buy tax:          3%
User receives:  970 CALIBER
```

Direct Pharaoh liquidity additions have the same problem. The router computes optimal amounts, transfers those exact amounts, and mints LP:

```
(amountA, amountB) =
    _addLiquidity(tokenA, tokenB, stable, amountADesired, amountBDesired, amountAMin, amo
        untBMin);

_safeTransferFrom(tokenA, msg.sender, pair, amountA);
_safeTransferFrom(tokenB, msg.sender, pair, amountB);
liquidity = IPair(pair).mint(to);
```

If one side is CALIBER and the pair is taxed, the pair receives less CALIBER than the router expects. LP is minted from actual balances received, so the user can receive less LP than expected and may donate excess quote token into the pool.

This is why AmmoLiquidityManager exists, but it also means normal Pharaoh liquidity flows are not safe for CALIBER users.

Impact

- Standard CALIBER sells through normal Pharaoh router functions can revert.
- Standard CALIBER buys can succeed while delivering less CALIBER than the user's effective minimum output.
- Direct liquidity adds can mint less LP than expected or donate value to the pool.

Recommendation

- Provide an CALIBER-specific swap adapter or frontend integration that always uses supporting fee-on-transfer router functions for CALIBER routes.
- Keep direct Pharaoh liquidity adds disabled or route users through `AmmoLiquidityManager` or a tax-aware liquidity wrapper.

Ammo: Acknowledged. Pharaoh router tax incompatibility is treated as acceptable integration behavior; users should use `AmmoLiquidityManager`. The team also intends to use its own router by forking Pharaoh's router/pair setup and using fee-on-transfer-supporting functions for taxed-pool swaps.

Slayer Security: Acknowledged.

[M-02] Out-of-Order Fulfillments Can Overwrite Fresh Oracle Prices

Summary

`AmmoPriceFunctions` records the latest Chainlink Functions request ID in `lastRequestId`, but `_fulfillRequest()` accepts any successful callback. An older delayed request can therefore be fulfilled after a newer request and overwrite

fresh oracle prices with stale data.

Vulnerability Detail

When a price update is requested, the contract stores the latest request ID:

```
bytes32 requestId = _sendRequest(req.encodeCBOR(), subscriptionId, callbackGasLimit, donI
    d);
lastRequestId = requestId;
```

However, the fulfillment callback does not verify that the incoming `requestId` is still the latest request:

```
function _fulfillRequest(bytes32 requestId, bytes memory response, bytes memory err) inte
    rnal override {
    if (err.length > 0) {
        emit PriceUpdateFailed(requestId, err);
        return;
    }

    uint256[] memory prices = abi.decode(response, (uint256[]));
    oracle.setBatchPrices(marketAddresses, prices);
    emit PriceUpdateFulfilled(requestId, prices);
}
```

A stale callback can overwrite newer prices:

```
Request A observes price: $0.20
Request B observes price: $0.40
Request B fulfills first  -> oracle price becomes $0.40
Request A fulfills later -> oracle price becomes stale $0.20
```

Because `PriceOracle` timestamps writes at execution time, stale observations can appear fresh to dependent mint logic.

Impact

- Users can mint or redeem against stale prices that appear current on-chain.
- If a stale lower price overwrites the correct price, users may receive more AmmoTokens than intended for the same USDC deposit.
- Delayed fulfillments are realistic during oracle delays, network congestion, keeper disruption, or chain downtime.
- For example, if Automation submits **Request A** just before an Avalanche outage, the DON may not deliver the callback until after the chain resumes. By then prices may have moved, so Automation submits **Request B** with fresher data and `lastRequestId` advances to B. If **Request A** fulfills afterward anyway, it overwrites the oracle with pre-outage prices even though B was the latest

on-chain request. Avalanche has experienced outages as well - https://blockworks.com/news/avalanche-blockchain-downtime

Recommendation

- Reject stale callbacks by requiring the fulfilled request to match the latest request:

```
if (requestId != lastRequestId) {
    return;
}
```

Ammo: Acknowledged. No longer applicable because Chainlink Functions is no longer used.

Slayer Security: Acknowledged.

[M-03] Locked Request Pricing Can Be Abused Around Keeper Price Updates

Summary

`CaliberMarket` locks the oracle price when a user creates a mint or exit request. This design exposes the protocol to selective timing around keeper `setPrice` updates: users can front-run pending price updates, lock a favorable stale price, and keep the option until the keeper finalizes the request.

Vulnerability Detail

Mint requests store the current oracle price at request time:

```
function startMint(uint256 usdcAmount, uint64 deadline) external returns (uint256 orderId
    ) {
    (uint256 price, ) = _freshPrice();
    // ...
    mintOrders[orderId] = MintOrder({
        user: msg.sender,
        usdcAmount: usdcAmount,
        requestPrice: price,
        // ...
        status: OrderStatus.Requested
    });
}
```

`finalizeMint()` does not read the current oracle price. It mints using the stored `requestPrice`:

```
uint256 tokenAmount = _tokensForUsdc(netUsdc, order.requestPrice);
uint256 actualUsdc = _usdcForTokens(tokenAmount, order.requestPrice);
```

Exit requests similarly store the payout at request time:

```
(uint256 price, ) = _freshPrice();
(uint256 payoutUsdc, uint256 feeUsdc) = _exitQuote(tokenAmount, price, exitFeeBps);

exitOrders[orderId] = ExitOrder({
    user: msg.sender,
    tokenAmount: tokenAmount,
    requestPrice: price,
    payoutUsdc: payoutUsdc,
    exitFeeUsdc: feeUsdc,
    // ...
    status: OrderStatus.Requested
});
```

`finalizeExit()` pays the stored payout:

```
exitLiquidityPool.payExit(order.user, order.payoutUsdc);
```

If a user sees a pending keeper price update, they can submit a request before the update is mined:

```
Mint side:
Current oracle price = $1.00
Pending setPrice     = $1.10
User front-runs with startMint()
Order locks          = $1.00
Final mint occurs after oracle updates to $1.10
```

The reverse applies on exits:

```
Exit side:
Current oracle price = $1.00
Pending setPrice     = $0.90
User front-runs with requestExit()
Order locks payout   = $1.00
Final exit pays stale high value after oracle updates to $0.90
```

This is not ordinary random price drift. The user can selectively open requests only when pending or known off-chain price movement is favorable, turning the request window into a free option against the protocol.

Impact

- Users can lock stale favorable prices around keeper updates.
- The protocol can absorb losses on mint when real ammo prices rise after the request is opened.
- Existing holders can lock inflated exit payouts before a price decrease is mined.
- Loss scales with order size and the price movement between request creation and finalization.

Recommendation

- Price mints and exits at finalization using `_freshPrice()`, not the oracle value at request time.
- If request-time quotes must remain, use the worse price for the protocol: `max(requestPrice, currentPrice)` on mint and `min(requestPrice,` currentPrice) `on exit.`
- Cancel or require user confirmation when price moves beyond a configured threshold.

Ammo: Acknowledged. Request-time pricing around keeper updates is accepted as a protocol tradeoff.

Slayer Security: Acknowledged

[M-04] Permissionless Tax Sales Can Be Forced Through a Short TWAP Window

Summary

Any user can trigger CALIBER's accumulated tax sale with a dust wallet-to-wallet transfer once the token contract balance exceeds `taxSwapThreshold`. The sale uses a Pharaoh/Solidly pair `current()` TWAP quote for slippage protection, which can be forced into a very short manipulated window.

Vulnerability Detail

`_sellTaxes()` is called from `_transfer()` when the current transfer has no pool tax:

```
taxAmount = _determineTax(from, to, amount);

if (taxAmount == 0 && _shouldSwap()) {
    _sellTaxes();
}
```

Pool buys and sells accumulate tax inventory but do not trigger `_sellTaxes()`, because those transfers have `taxAmount > 0`:

```
if (taxAmount > 0) {
    balanceOf[address(this)] += taxAmount;
    emit Transfer(from, address(this), taxAmount);
}
```

Tax inventory can therefore sit above `taxSwapThreshold` until a normal wallet transfer occurs. A dust transfer such as `ammo.transfer(otherEOA, 1)` can flush the full accumulated balance.

The full tax balance is sold, and `amountOutMin` is derived from the Pharaoh pair's `Pair.current()` TWAP:

```
uint256 tokenBalance = balanceOf[address(this)];
uint256 twapOut = _taxSwapTwapOut(router, wavax_, swapPath.stable, tokenBalance);
uint256 amountOutMin = (twapOut * (_BPS_DIVISOR - slippageBps)) / _BPS_DIVISOR;

IDexRouter(router).swapExactTokensForETHSupportingFeeOnTransferTokens(
    tokenBalance,
    amountOutMin,
    routes,
    treasury_,
    block.timestamp
);
```

The pair's `sync()` is permissionless. After enough time has passed since the last observation, an attacker can call `sync()` to push a fresh observation, move the pool price, and wait for the next block. `Pair.current()` then measures only the short interval after that fresh observation.

Attack Example

Assume:

```
Official pair:       CALIBER / WAVAX
Tax balance:         100,000 CALIBER
taxSwapThreshold:    50,000 CALIBER
Pool liquidity:      shallow enough for the tax sale to move price
```

Block N

1. At least one observation period has passed.
2. The attacker calls `pair.sync()` to push a fresh observation.
3. The attacker sells CALIBER into the official pair and depresses the CALIBER price.
4. The pool sell may add more tax inventory, but does not trigger `_sellTaxes()`.

Block N+1

1. The attacker sends a dust transfer between normal addresses:

```
ammo.transfer(otherEOA, 1);
```

2. The transfer has `taxAmount == 0`, so `_shouldSwap()` is checked.
3. `_sellTaxes()` runs and sells the full accumulated tax balance.
4. `Pair.current()` reads the short manipulated window.
5. The protocol sells into the already-depressed pool and receives less WAVAX.
6. The attacker can buy back after the forced sale if the extra price impact covers taxes, swap fees, and cross-block risk.

This is strongest when the accumulated tax balance is large relative to official-pair liquidity.

Impact

- Treasury tax inventory can be sold at a manipulated unfavorable price.
- The treasury receives less WAVAX for the same collected CALIBER taxes.
- Any user can choose when to flush the tax balance with a dust transfer.

Recommendation

Remove the automatic `_sellTaxes()` call from `_transfer()`. DEX trades should only accumulate tax inventory; liquidation should be a deliberate keeper action.

Expose a keeper-only entry point that accepts a minimum output floor. The keeper sets `minAmountOut` off-chain (for example from a full TWAP quote minus an operational slippage buffer) and reverts if the swap would deliver less.

Ammo: Fixed: commit [df4e5c22ac60b53a20512f201faf073bb58409fd](). The tax is now sent directly to the protocol treasury instead of being sold directly.

Slayer Security: Verified.

[M-05] First Pool Updated Near Farm Cap Can Starve Other Rewardable Pools

Summary

`AmmoMarketLPFarm` splits emissions equally across rewardable pools, but applies `farmMintCap` inside each individual `updatePool()` call. Near cap exhaustion, the first pool updated can consume the remaining cap and leave other pools with zero rewards for the same elapsed reward window.

Vulnerability Detail

`updatePool()` calculates one pool's equal-share reward and immediately caps it:

```
uint256 poolReward = _capFarmAccrual(
    _emitted(pool.lastRewardTime, current) / rewardableCount
);
```

`_capFarmAccrual()` consumes the global remaining cap during that single pool update:

```
uint256 remaining = farmMintCap - totalFarmAccrued;
if (amount > remaining) {
    amount = remaining;
}

totalFarmAccrued = totalFarmAccrued + amount;
```

If two pools were rewardable for the same period but only one pool-share remains under the cap, the first updated pool receives the full remaining cap:

```
Remaining farm cap: 475 CALIBER
Elapsed emission:   950 CALIBER
Rewardable pools:   2
Equal share:        475 CALIBER per pool

updatePool(0) -> receives 475 CALIBER and exhausts cap
updatePool(1) -> receives 0 CALIBER
```

The final capped reward distribution becomes update-order dependent instead of equal across rewardable pools.

Impact

- Rewards near the farm cap boundary are first-come-first-served.
- Users in later-updated pools can receive zero rewards for an interval where they were active and staked.
- Any user can favor a pool by calling `updatePool()`, `harvest()`, or another state-changing path for that pool first when the farm is near the cap.

Recommendation

- Apply the farm cap once at the global reward-window level, then split the capped amount across rewardable pools.
- Update all affected pools from the same capped global amount, preserving the equal-pool distribution rule.

- If first-come-first-served cap exhaustion is intended, document it explicitly.

Ammo: Valid. The protocol has removed the farm contract altogether and switched to a points system.

Slayer Security: Acknowledged.

[L-01] Refunded Mint Dust Is Still Counted Toward Protocol Token Release

Summary

`finalizeMint()` refunds unused USDC dust to the user, but still calls `recordCaliberMint()` with the full original mint order amount. This can slightly overstate protocol mint volume and release protocol tokens based on USDC that was not actually used for the token purchase.

Vulnerability Detail

During finalization, the contract computes the USDC actually used and refunds the leftover dust:

```
uint256 actualUsdc = _usdcForTokens(tokenAmount, order.requestPrice);
uint256 refund = netUsdc - actualUsdc;

_safeTransfer(usdc, treasury, actualUsdc);
if (refund > 0) {
    _safeTransfer(usdc, order.user, refund);
}
```

However, protocol token release is recorded using the original `order.usdcAmount`:

```
token.mint(order.user, tokenAmount);
emissionController.recordCaliberMint(order.usdcAmount);
```

`recordCaliberMint()` increases global volume from the passed amount:

```
globalUsdcVolume += usdcAmount;
```

As a result, refunded dust is counted as demand even though it was returned to the user.

Impact

- Protocol token release can be slightly accelerated by refunded dust.

- Reported global USDC volume can be overstated relative to USDC actually used to mint AmmoTokens.
- The impact is limited to rounding dust but can accumulate across many mints.

Recommendation

- Record only the USDC that was actually used for the mint, plus any retained fee if protocol-token release is intended to include fees:

```
uint256 recordedUsdc = feeAmount + actualUsdc;
emissionController.recordCaliberMint(recordedUsdc);
```

Ammo: Fixed.

Slayer Security: Verified.

[L-02] Expired-User Authorization Uses Misleading Revert Errors

Summary

`_requireKeeperOrExpiredUser()` uses `NotKeeper` and `DeadlineExpired` in cases where the caller is not the order user or the deadline has not expired yet. These error names describe the opposite or only part of the actual failure condition, making integrations and debugging misleading.

Vulnerability Detail

The helper allows keepers immediately, or the order user only after the deadline has passed:

```
function _requireKeeperOrExpiredUser(address user, uint64 deadline) internal view {
    if (!manager.isKeeper(msg.sender)) {
        if (msg.sender != user) revert NotKeeper();
        if (deadline == 0) revert DeadlineNotSet();
        if (block.timestamp <= deadline) revert DeadlineExpired();
    }
}
```

The first revert is not strictly a keeper failure; it is an unauthorized caller failure. The final revert occurs when the deadline has **not** expired, but the error says `DeadlineExpired`.

Impact

- Frontends, monitoring, and tests can misinterpret why cancellation failed.
- Users may see an expired-deadline error when the actual issue is that the deadline is still active.

- The issue does not directly affect funds, but it reduces operational clarity.

Recommendation

- Replace the errors with names that match the failed condition:

```
if (msg.sender != user) revert UnauthorizedCaller();
if (deadline == 0) revert DeadlineNotSet();
if (block.timestamp <= deadline) revert DeadlineNotExpired();
```

Ammo: Fixed.

Slayer Security: Verified.

[L-03] Emergency Withdraw Consumes Farm Cap Without Minting Rewards

Summary

`AmmoMarketLPFarm` increments `totalFarmAccrued` during `updatePool()` for the full pool reward, including rewards belonging to users who later call `emergencyWithdraw()` and forfeit those rewards. Since forfeited rewards are never minted, `totalFarmAccrued` can reach `farmMintCap` before the protocol has actually paid out the full cap.

Vulnerability Detail

`updatePool()` accrues the full pool reward and immediately counts it against the global farm cap:

```
uint256 poolReward = _capFarmAccrual(
    _emitted(pool.lastRewardTime, current) / rewardableCount
);
pool.accRewardPerShare += (poolReward * ACC_REWARD_PRECISION) / pool.totalStaked;
```

`_capFarmAccrual()` advances `totalFarmAccrued`:

```
totalFarmAccrued = accrued + amount;
```

Actual tokens are minted only when a user harvests:

```
emissionController.mintFarmReward(to, pending);
```

However, `emergencyWithdraw()` updates the pool and then skips `_harvest()`:

```
function emergencyWithdraw(uint256 pid) external nonReentrant {
    updatePool(pid); // totalFarmAccrued increases for the full pool reward

    user.amount = 0;
    user.rewardDebt = 0;
    pool.totalStaked -= amount;
    _safeTransfer(pool.stakingToken, msg.sender, amount);
}
```

The withdrawn user's reward share is counted against `farmMintCap` but is never minted:

```
totalFarmAccrued += full pool reward
emergencyWithdraw forfeits user's pending reward
mintFarmReward is never called for the forfeited amount
remaining farm budget is reduced anyway
```

Impact

- The farm can stop emitting before `farmMintCap` tokens have actually been minted.
- Honest remaining stakers can lose rewards because forfeited emergency-withdraw rewards consumed cap budget.
- The loss scales with the amount of unharvested rewards forfeited through emergency withdrawals.

Recommendation

In `emergencyWithdraw()`, compute the user's forfeited pending rewards using the same logic as `_harvest()`, subtract that amount from `totalFarmAccrued`, then reset `user.amount` and `rewardDebt`. That keeps the farm cap tied to rewards that were minted or remain claimable, instead of counting forfeited accruals as spent budget.

Ammo: Intended design. Emergency withdraw forfeiting rewards and consuming farm cap is intended behavior.

Slayer Security: Acknowledged.

[L-04] Dust Exits Can Create Unfinalizable Orders

Summary

`requestExit()` accepts any nonzero `tokenAmount`, but very small amounts can round down to `payoutUsdc == 0`. The exit request is still stored and tokens are transferred in, but `finalizeExit()` later reverts because

`ExitLiquidityPool.payExit()` rejects zero-value payments.

Vulnerability Detail

`requestExit()` only rejects a zero token input:

```
if (tokenAmount == 0) revert InvalidAmount();
```

It then computes:

```
(uint256 payoutUsdc, uint256 feeUsdc) = _exitQuote(tokenAmount, price, exitFeeBps);
```

For dust amounts, `_usdcForTokens()` can floor to zero, so `payoutUsdc` is zero while the order is still created.

During finalization, the market attempts to pay out:

```
exitLiquidityPool.payExit(order.user, order.payoutUsdc);
```

`ExitLiquidityPool` reverts on zero payment amount:

```
if (amount == 0) revert InvalidAmount();
```

Impact

- Dust exit orders become unfinalizable and waste keeper gas on failed finalize attempts.
- The protocol accrues avoidable operational overhead and stale requested-exit state.
- If a user sets `deadline = 0`, cancellation is not user-triggerable and depends on keeper intervention.

Recommendation

- Reject zero-payout exits at request creation:

```
(uint256 payoutUsdc, uint256 feeUsdc) = _exitQuote(tokenAmount, price, exitFeeBps);
if (payoutUsdc == 0) revert InvalidAmount();
```

Ammo: Acknowledged.

Slayer Security: Acknowledged.

8. Findings (Phase - 2)

[M-01] Non-Native Pair Liquidity Removal Can Bypass Effective Slippage Checks

Summary

The team addressed Phase - 1 M-01 for supported swap and native-token liquidity flows by moving toward its own Pharaoh router/pair setup and fee-on-transfer-supporting swap functions. However, this does not address non-native pair liquidity removal if users interact with unsupported pairs directly through a standard router path.

For a pair such as `Caliber/USDC`, a standard `removeLiquidity` call can pass the user's minimum output checks using the gross amounts returned by the pair burn, while the user receives less Caliber after transfer tax.

Vulnerability Detail

Assume a `Caliber/USDC` pair where:

```
Token A = Caliber
Token B = USDC
User LP balance = 1,000 LP tokens
Caliber transfer tax = 5%
```

The user calls `removeLiquidity` with:

```
minTokenA = 980 Caliber
minTokenB = 200 USDC
```

When the LP tokens are burned, the pair reports the gross withdrawal amounts:

```
amountA = 1,000 Caliber
amountB = 200 USDC
```

The router checks slippage against those gross values:

```
amountA >= minTokenA -> 1,000 >= 980, passes
amountB >= minTokenB -> 200 >= 200, passes
```

However, the Caliber transfer from the taxed pool to the user is still subject to transfer tax. With a 5% tax, the user receives:

```
950 Caliber
200 USDC
```

The transaction succeeds even though the user's effective Caliber output is below their intended minimum of `980`. This happens because the router validates the burn-returned amount, not the net amount actually received by the user after transfer tax.

Impact

Users removing liquidity from non-native Caliber pairs can receive fewer Caliber tokens than their configured minimum while the transaction still succeeds. This weakens slippage protection for direct non-native pair liquidity removal.

Recommendation

- Update the router or liquidity manager so removal checks the user's net received amount after transfer tax, or otherwise uses a fee-on-transfer-aware liquidity-removal path.

Ammo: Acknowledged. The team does not plan to support non-native pairs at this time. Liquidity support will remain focused on native-token pairs for now, and if support for additional pair types becomes necessary, the team may update or replace the router.

Slayer Security: Acknowledged.

[L-01] Denied Spender Can Move Tokens via Existing Allowances

Summary

`CaliberToken` deny-list enforcement checks only `from` and `to` inside `_transfer`. It does not check the `spender` in `transferFrom`, so a denied address with an existing allowance can still move a holder's tokens to any non-denied recipient.

Vulnerability Detail

`CaliberToken` deny-list enforcement only checks the `from` and `to` addresses inside `_transfer`. The `spender`, `msg.sender`, is not checked in `transferFrom`.

As a result, a denied address with an existing allowance from a non-denied holder can still call `transferFrom(holder, recipient, amount)` and move the holder's tokens to any non-denied recipient.

```
function transferFrom(address from, address to, uint256 amount) external returns (bool) {
    uint256 allowed = allowance[from][msg.sender];
    if (allowed < amount) revert InsufficientAllowance();

    if (allowed != type(uint256).max) {
        allowance[from][msg.sender] = allowed - amount;
        emit Approval(from, msg.sender, allowance[from][msg.sender]);
    }

    _transfer(from, to, amount);
    return true;
}

function _transfer(address from, address to, uint256 amount) internal {
    if (to == address(0)) revert ZeroAddress();
    if (balanceOf[from] < amount) revert InsufficientBalance();

    if (manager.isDenied(from) || manager.isDenied(to)) revert Denied();

    ...
}
```

For comparison, USDC-style blacklist implementations commonly check the sender as well as the source and destination addresses:

https://basescan.org/address/0x2ce6311ddae708829bc0784c967b7d77d19fd779#code#F14#L266

Flow

1. Alice approves Bob:

`approve(Bob, 1,000 CALIBER)`

2. Bob is later added to the deny-list.
3. Bob calls:

`transferFrom(Alice, Carol, 1,000 CALIBER)`

4. `_transfer` checks:

`isDenied(Alice) == false`

`isDenied(Carol) == false`

5. The transfer succeeds, even though Bob is denied.

Impact

The direct impact is limited because the denied address must already have an allowance from a non-denied token holder. One realistic scenario is that a victim approves a phishing contract, malicious contract, or bridge contract, and Ammo later deny-lists that spender to stop malicious token movement, a bridge-related attack, or laundering activity. Because `transferFrom` does not check `msg.sender`, the denied spender could still move the victim's approved tokens despite being

deny-listed.

This is a known purpose of blacklist controls: preventing scam contracts, compromised bridges, or malicious actors from continuing to move tokens. However, the scenario depends on a prior approval mistake by the token holder or integration, and it is unlikely to create significant protocol-level impact on its own. For that reason, this finding is assessed as Low severity rather than High or Medium.

The fix should still be implemented because it is simple, aligns the deny-list behavior with user expectations, and closes a potential issue that could become more relevant in future integrations.

Recommendation

- Enforce the deny-list on `msg.sender` in `transferFrom`:

```
if (manager.isDenied(from) || manager.isDenied(to) || manager.isDenied(msg.sender)) rever
    t Denied();
```